

# NoSQL — Unit 3 Notes

Module 1-4: Map-Reduce, Key-Value Databases, Use Cases & Anti-Patterns, and Exam Preparation  
(Based on NoSQL Distilled — Sadalage & Fowler, Chapters 7 & 8)

## MODULE 1: MAP-REDUCE

### Why Map-Reduce Is Needed

Running a database on a cluster changes the rules not just for storing data, but also for **processing** it. With a single centralized database, you can run processing logic either on the server or on a client machine. But once data is spread across a cluster, hauling all of it across the network to one place for processing becomes impractical.

The **map-reduce pattern** (a form of the "Scatter-Gather" pattern) organizes processing so that a cluster's many machines can be used in parallel, while keeping as much processing as possible on the same node that holds the data. It first gained prominence through Google's MapReduce framework and is widely implemented today via the open-source Hadoop project, as well as built directly into several NoSQL databases. The name "map-reduce" borrows from the **map** and **reduce** operations found in functional programming languages.

**Exam Point:** Map-reduce exists to solve a specific tension: aggregate-oriented databases are excellent for retrieving one aggregate at a time, but poor for queries that must scan and summarize data **across** many aggregates spread over many machines.

### Basic Map-Reduce

**The motivating scenario:** Suppose we have chosen *orders* as our aggregate, with each order containing line items (product ID, quantity, price charged). This aggregate makes sense because people usually want to see a whole order in one access. Since we have a huge number of orders, the dataset has been sharded across many machines.

Now suppose the sales analysis team wants a report of **each product and its total revenue over the last seven days**. This report does not fit the aggregate structure at all — to build it, you would have to visit every machine in the cluster and examine many records on each one. This is exactly the situation map-reduce is designed for.

### The Map Stage

The first stage of a map-reduce job is the **map**. A map is a function whose **input is a single aggregate** and whose **output is a set of key-value pairs**. In the order example, the input is an order, and the output would be key-value pairs corresponding to each line item — the **product ID** as the key, and an embedded map containing quantity and price as the value.

**Crucial property:** Each application of the map function is completely **independent** of all the others. This means map tasks are safely parallelizable — the framework can create efficient map tasks on each node and freely allocate orders to map tasks, giving both parallelism and locality of data access (the map runs right where the data already lives).

### The Reduce Stage

A map function only ever operates on a **single record**. The **reduce** function is what takes multiple map outputs that share the same key and **combines their values** into one. For example, a map stage might yield 1,000 line items for the product "Database Refactoring"; the reduce function reduces these down to a single record holding the totals for quantity and revenue.

The map-reduce framework arranges for map tasks to run on the nodes that hold the relevant data, and then moves the resulting key-value pairs to wherever the reduce function runs. To make writing the reduce function easier, the framework **collects all the values for a given key** and calls the reduce function once, passing it the key and the full collection of values for that key.

### Map vs. Reduce: Side-by-Side

| Stage         | Input                                                               | Output                                                               | Key Property                                                                      |
|---------------|---------------------------------------------------------------------|----------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| <b>Map</b>    | A single aggregate (e.g., one order)                                | A set of key-value pairs (e.g., product ID → quantity/price)         | Independent of all other map calls — fully parallelizable, runs local to the data |
| <b>Reduce</b> | A single key plus the collection of all values emitted for that key | One summarized value for that key (e.g., total quantity and revenue) | Can use all values for a key, but is limited to operating within a single key     |

**Real-world intuition:** Think of the map stage as thousands of clerks, each opening one order file and jotting down "Product X: sold 2 units for ₹500" on a slip of paper — working totally independently and in parallel. The reduce stage is then like a set of accountants, each responsible for one specific product, who collect every slip mentioning their product and add up the totals.

**Exam Point:** To run a map-reduce job, you only need to write **two functions** — map and reduce. The map function's independence is what makes map-reduce naturally parallel and cluster-friendly.

## Partitioning and Combining

In its simplest form, a map-reduce job has a single reduce function, and the outputs of all map tasks are concatenated together before being sent into that one reduce. This works, but it leaves performance on the table — there is more we can do to increase parallelism and reduce data transfer.

### Partitioning

Each reduce function call operates on the results for a **single key**. This is a limitation (you cannot do anything in a reduce that operates across keys), but it is also an opportunity: since each key's reduction is independent of every other key's, we can run **multiple reducers in parallel**.

To exploit this, the mapper's output on each node is divided up based on key, with multiple keys typically grouped together into **partitions**. The framework then gathers, for a given partition, all the data produced across every node, combines it into a single group, and sends it to one reducer. Multiple reducers can then work on different partitions simultaneously, and their final results are merged. This step is also called "**shuffling**," and partitions are sometimes called "**buckets**" or "**regions**."

### Combining

The second optimization addresses the sheer **volume of data** moved between the map and reduce stages — much of it repetitive key-value pairs sharing the same key. A **combiner function** cuts this down by combining all the data for the same key into a single value **before** it is sent across the network.

A combiner function is, in essence, a reducer function — in many cases, the very same function used for the final reduction can also be used as the combiner. For this to work, the reduce function's **output must match its input in shape** — such a function is called a **combinable reducer**.

**Not every reducer is combinable.** Consider a reduce function that counts the number of **unique** customers who ordered a particular product. Its output (a count) is a different shape from its input (product-customer pairs), so it cannot double as a combiner — although a separate, simpler combining step (eliminating duplicate product-customer pairs) can still be used before the final reduce.

When reducers are combinable, the map-reduce framework can run them not only in parallel across different partitions, but also **in series** — reducing the same partition at different times and places, and even starting to combine before all mappers have finished. Some map-reduce frameworks require every reducer to be combinable to maximize this flexibility; non-combinable logic must then be broken into separate, pipelined map-reduce steps.

| Optimization | What It Solves                                        | How It Works                                                                                                                   |
|--------------|-------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Partitioning | Limited parallelism if there's only one reducer       | Groups keys into partitions so multiple reducers can run in parallel, each on its own partition                                |
| Combining    | Excessive data transfer between map and reduce stages | Pre-aggregates repeated key-value pairs on the same node before shipping data across the network, using a "combinable reducer" |

**Real-world intuition:** Partitioning is like assigning "Product A" totals to Accountant 1 and "Product B" totals to Accountant 2, so they work simultaneously instead of queueing behind one accountant. Combining is like each clerk pre-summing their own slips for a product before handing them to the accountant, so the accountant receives one number per clerk instead of a thousand individual slips.

**Exam Point:** Partitioning increases **parallelism** across keys; combining reduces **network data transfer** by pre-summarizing per-key data before the shuffle. A combiner is valid only when the reduce function's output has the same shape as its input.

## Composing Map-Reduce Calculations

Map-reduce trades away flexibility in how you structure a computation in exchange for a straightforward way to parallelize it across a cluster. Because of this trade-off, calculations must be structured to fit map-reduce's constraints: within a map task you can only operate on a **single aggregate**, and within a reduce task you can only operate on a **single key**.

## The Averages Problem

A key limitation is that calculations must be built from operations that compose well with "reduce." **Averages are not composable** — if you have two groups of orders, you cannot combine their two averages directly to get the overall average. Instead, you must combine the **total sum** and the **count** from each group first, and only then calculate the average from the combined sum and count.

**Similarly for counts:** to count records, the map function emits a value of **1** for each record, and these 1's are simply summed in the reduce stage to get the total count.

**Exam Point:** Averages are **not composable** across a distributed reduce — you must carry forward the sum and the count separately through the reduce stage, and only compute the final average at the very end.

## A Two-Stage Map-Reduce Example

As calculations grow more complex, it helps to break them down into **stages** using a pipes-and-filters approach — much like piping commands together in UNIX — where the output of one map-reduce stage becomes the input to the next.

**Worked example:** Suppose we want to compare product sales for each month of 2011 against the same month in the prior year (2010).

| Stage            | What It Does                                                                                                                                                                                                   |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Stage 1          | Reads the original order records and produces key-value pairs for the sales of each product, per month — using a <b>composite key</b> (product + month) so records can be reduced on multiple combined fields  |
| Stage 2 — Map    | Processes Stage 1's output depending on the year: a 2011 record populates the "current year quantity," a 2010 record populates the "prior year quantity." Records from earlier years (e.g., 2009) emit nothing |
| Stage 2 — Reduce | Merges the incomplete records by summing values, combining the current-year and prior-year partial records into one complete record — with the year-on-year comparison calculated from the combined values     |

Breaking a report down into multiple map-reduce steps makes it far easier to write than cramming all the logic into a single step. It also means the **intermediate output** can potentially be reused for other reports, saving both programming and execution time. When intermediate results are reused this way, they can be stored in the data store as a **materialized view** — and because early map-reduce stages usually involve the heaviest data access, saving them is especially valuable as a foundation for many downstream calculations.

**Exam Point:** Composing map-reduce calculations into multiple pipelined stages (output of one reduce feeding the map of the next) makes complex reports easier to build, and lets intermediate results be reused as materialized views.

## Incremental Map-Reduce

The examples discussed so far are **complete** map-reduce computations — starting from raw inputs and producing a final output from scratch each time. But many map-reduce jobs take a long time, even on a cluster, and new data keeps arriving, meaning the computation would otherwise need to be rerun constantly just to stay up to date. **Incremental map-reduce** restructures the computation so that only the minimum necessary recomputation is done.

| Stage  | How Incremental Updates Work                                                                                                                                  |
|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Map    | Easy to handle incrementally — since each map call is isolated, the mapper only needs to be rerun for aggregates whose <b>input data has actually changed</b> |
| Reduce | More complex, since it pulls together outputs from many maps and any change in those outputs could in principle trigger a new reduction                       |

### Reducing the recomputation burden:

- If the data is **partitioned** for reduction, a partition that has not changed does not need to be re-reduced at all.
- If there is a **combiner step** and its source data hasn't changed, it does not need to be rerun either.
- If the reducer is **combinable** and changes are purely **additive** (only new records are being added, nothing is changed or deleted), the reduce can simply be run using the existing result plus the new additions — no full recomputation needed.
- If there are **destructive changes** (updates or deletes), some recomputation can still be avoided by breaking the reduce operation into smaller steps and only recalculating the steps whose inputs actually changed — essentially treating the computation as a dependency network.

**Real-world intuition:** A retailer's "total sales per product this year" materialized view doesn't need to be recomputed from all historical orders every night — if only new orders came in today (an additive change), the view can simply be updated by adding

today's totals to yesterday's stored result, rather than re-scanning the entire year's data.

**Exam Point:** Map stages are trivially incremental because each map call is isolated. Reduce stages can be made incremental too — especially when changes are purely **additive** and the reducer is **combinable**, in which case you can simply combine the old result with the new data instead of recomputing from scratch.

## MODULE 2: KEY-VALUE DATABASES

### What Is a Key-Value Store

A key-value store is essentially a **simple hash table**, used primarily when all access to the database happens via a **primary key**. Think of a two-column table in a traditional RDBMS — an ID column and a NAME column — where ID is the key and NAME holds the value. The application provides an ID and a value and persists the pair; if the ID already exists, the current value is overwritten, otherwise a new entry is created.

**The API is deliberately minimal.** Key-value stores are the simplest kind of NoSQL data store from an API perspective — the client can only:

| Operation | What It Does                                     |
|-----------|--------------------------------------------------|
| Get       | Retrieve the value stored for a given key        |
| Put       | Store (or overwrite) a value for a given key     |
| Delete    | Remove a key (and its value) from the data store |

The value itself is just a **blob** — the data store stores it without caring or knowing what's inside; interpreting that value is entirely the application's responsibility. Because access is always by primary key, key-value stores generally offer **great performance** and can be scaled easily.

**Popular key-value databases:** Riak, Redis (sometimes called a "data structure server"), Memcached (and its variants), Berkeley DB, HamsterDB (suited for embedded use), Amazon DynamoDB, and Project Voldemort (an open-source implementation inspired by Dynamo). In some stores such as Redis, the stored aggregate doesn't have to be a plain domain object — Redis can store lists, sets, and hashes, and supports range, diff, union, and intersection operations, making it usable in more ways than a standard key-value store.

### Organizing Keys: Buckets

Using Riak as a representative example, keys are stored inside **buckets** — simple flat namespaces that segment keys, similar in spirit to tables. If we stored session data, shopping cart data, and user preferences all under a single key in a single bucket, we would risk **key conflicts** because different types of aggregates share the same namespace. Two common fixes:

- **Compound keys** — appending the object type to the key itself (e.g., `288790b8a421_userProfile`) to segment data within one bucket.
- **Domain buckets** — separate buckets for separate object types (e.g., a `UserProfile` bucket and a `ShoppingCart` bucket), letting the client driver handle serialization/deserialization and letting you read exactly the object you need without changing your key design.

**Exam Point:** A key-value store never inspects the value — it is an opaque blob to the database. All the "intelligence" about what's inside a value, and how to avoid key collisions, lives entirely in the application's key design.

### Key-Value Store Features

#### Consistency

Consistency in a key-value store is meaningful only for operations on a **single key**, since every operation is just a get, put, or delete on one key. Optimistic writes are possible but expensive to implement, because the data store has no way of understanding whether a value has meaningfully "changed."

Distributed key-value stores like Riak typically implement an **eventually consistent** model. Since a value may already be replicated to other nodes, conflicting updates are resolved in one of two ways:

| Conflict-Resolution Strategy | How It Works                                                              |
|------------------------------|---------------------------------------------------------------------------|
| Newest write wins            | The most recent write is kept, and older conflicting writes are discarded |

|                              |                                                                                                               |
|------------------------------|---------------------------------------------------------------------------------------------------------------|
| Return all values (siblings) | All conflicting values are returned to the client, and the client (application) decides how to reconcile them |
|------------------------------|---------------------------------------------------------------------------------------------------------------|

These options are configured when a bucket is created — for example, setting the required write acknowledgment count (**W**) equal to the replication factor (**N**) forces every replica to be consistent before a write is considered successful, at the cost of write performance. An `allowSiblings` flag controls whether conflicting writes are allowed to coexist (creating siblings) or whether the last write simply wins.

## Transactions

Different key-value databases implement transactions very differently, and generally speaking, there are **no strong guarantees** on writes across multiple keys. Riak, for instance, uses the concept of a **quorum** via its `W` (write) value during the write API call.

**Worked example:** With a Riak cluster of replication factor  $N = 5$  and a required write quorum of  $W = 3$ , a write is only reported successful once at least three of the five nodes confirm it. This gives the cluster write tolerance of  $N - W = 2$  — the cluster can tolerate two nodes being down and still accept writes, though data on those two nodes would be temporarily missing for reads until they recover.

## Query Features

Key-value stores can be queried **by key — and that's essentially it**. If you need to query by some attribute inside the value, it generally isn't possible at the database level: your application would have to read the value itself and inspect it to check whether the attribute matches.

This creates a practical problem: what if you don't know the key, especially while debugging? Most key-value stores won't hand you a list of every key, and even if they did, fetching each value one-by-one to check its contents would be extremely cumbersome. Some databases work around this with add-on search capabilities — for example, **Riak Search**, which allows querying values much like a Lucene index.

Because of this, careful thought must go into **key design**: can the key be generated algorithmically? Supplied by the user (a user ID or email)? Derived from a timestamp or other external data? These query characteristics are exactly why key-value stores are natural fits for session data (session ID as key), shopping carts, and user profiles — and many stores support an **expiry** setting to automatically remove keys after a set time, which is especially useful for session or cart data.

## Structure of Data

Key-value databases are indifferent to what is actually stored in the value — it can be a blob, plain text, JSON, XML, or anything else. Some stores (like Riak) let you tag the value's content type in the request, but the database itself never inspects or enforces that structure.

## Scaling

Many key-value stores scale using **sharding**: the value of the key determines which node stores it (for example, sharding by the first character of the key). As more nodes are added, this can meaningfully boost performance — but it also means that if the node responsible for a given range of keys goes down, all data (and the ability to write new data) for that range becomes unavailable.

Stores such as Riak let you tune the three classic CAP-related parameters directly:

| Parameter | Meaning                                                                                           |
|-----------|---------------------------------------------------------------------------------------------------|
| N         | The number of nodes across which each key-value pair is replicated                                |
| R         | The number of nodes that must respond to a GET request before the read is considered successful   |
| W         | The number of nodes that must acknowledge a PUT request before the write is considered successful |

**Worked example:** In a 5-node Riak cluster, setting  $N = 3$  means all data is replicated to at least three nodes; setting  $R = 2$  means any two of those nodes must respond to a GET for it to succeed; setting  $W = 2$  means a PUT must be written to two nodes before it's considered successful. Tuning these values lets you trade off read availability against write availability according to your application's needs.

**Exam Point:** A key-value store's  $N$ ,  $R$ , and  $W$  settings are the same quorum concepts covered in Unit 2 — key-value databases are a concrete, practical example of tunable consistency in action.

# Suitable Use Cases

Because a key-value store offers fast, single-request access to an entire self-contained blob of data by key, it fits especially well wherever an application needs to store and retrieve one complete "thing" per unique identifier, with no need to query inside that thing.

## Storing Session Information

Every web session typically gets a unique **session ID**. Applications that would otherwise store session data on disk or in an RDBMS benefit greatly from a key-value store, because the **entire session** can be stored with a single PUT and retrieved with a single GET. Since everything about the session lives in one object, this is extremely fast. Memcached is widely used for this; Riak is a good choice when high availability of session data matters.

## User Profiles and Preferences

Almost every user has a unique user ID or username, along with preferences such as language, color theme, timezone, or which products they can access. All of this can be bundled into a single object, so fetching a user's preferences takes just **one GET operation**. The same approach works well for storing product profiles.

## Shopping Cart Data

E-commerce websites need shopping carts tied to a specific user, and want that cart to be available consistently across browsers, devices, and sessions. The entire cart can be stored as the value, keyed by user ID. A Riak-style cluster, with its emphasis on availability, is particularly well suited to this kind of workload, since a shopper should never lose their cart due to a single node's downtime.

| Use Case                    | Why It Fits                                                                                              |
|-----------------------------|----------------------------------------------------------------------------------------------------------|
| Session Information         | Entire session read/written in a single GET/PUT, keyed by session ID; can auto-expire                    |
| User Profiles / Preferences | All of a user's settings bundled into one object, fetched in a single GET keyed by user ID               |
| Shopping Cart Data          | Whole cart stored as one value keyed by user ID; benefits from high availability across devices/sessions |

**Real-world intuition:** An airline's website storing a logged-in user's seat-selection-in-progress as session data, or a grocery delivery app storing a shopper's entire cart, both want the same thing: one fast round trip to fetch or save "everything about this user's current activity" — exactly what a key-value GET/PUT is optimized for.

**Exam Point:** All three classic key-value use cases share the same shape: a single, self-contained object, retrieved whole, by a single well-known key — session ID, user ID, or cart owner's ID.

# When Not to Use Key-Value Stores

## Relationships among Data

If your application needs relationships between different sets of data, or needs to correlate data across different keys, key-value stores are not a good fit — even though some key-value stores offer limited "link-walking" features, this is not their strength.

## Multi-operation Transactions

If you are saving multiple keys and need the ability to **roll back** all of them if any single save fails, key-value stores are not the right solution — they offer no built-in multi-key transactional guarantee.

## Query by Data

If you need to search for records based on something found **inside the value** rather than the key, key-value stores will generally perform poorly, since the database cannot inspect the value itself. (A few products, like Riak Search, or integration with engines like Lucene or Solr, are exceptions.)

## Operations by Sets

Since every operation works on one key at a time, there is no way to operate on **multiple keys simultaneously** at the database level. Any such multi-key operation has to be handled entirely on the client/application side.

|  |  |
|--|--|
|  |  |
|--|--|

| Anti-Pattern                 | Why Key-Value Stores Struggle                                                                  |
|------------------------------|------------------------------------------------------------------------------------------------|
| Relationships among Data     | No native way to correlate or join data across different keys                                  |
| Multi-operation Transactions | No built-in rollback guarantee across multiple key writes                                      |
| Query by Data                | The database cannot inspect the value's contents to filter or search                           |
| Operations by Sets           | Only one key can be operated on at a time — set-based operations must be handled by the client |

**Real-world intuition:** A social network wanting to find "all friends of friends who live in Mumbai" cannot do this efficiently with a key-value store, because that query requires correlating data across many different keys (each user's friend list) rather than fetching one self-contained object by a single known key — this is precisely the kind of relationship-heavy query a graph or relational database handles far better.

**Exam Point:** The moment a requirement mentions "relationships," "joins," "search inside the data," "transactions spanning multiple records," or "querying by something other than the ID," it's a strong signal that a plain key-value store is the wrong tool.

## MODULE 4: UNIT 3 SUMMARY & EXAM PREPARATION

### 4.1 Key Definitions to Memorize

| Term                     | Definition                                                                                                                                      |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Map-Reduce               | A pattern for parallelizing computation across a cluster by splitting work into independent "map" and "reduce" stages                           |
| Map Function             | A function whose input is a single aggregate and whose output is a set of key-value pairs; fully independent and parallelizable                 |
| Reduce Function          | A function that takes all values emitted for a single key and combines them into one summarized output                                          |
| Partitioning (Shuffling) | Dividing mapper output by key into groups ("partitions"/"buckets") so multiple reducers can run in parallel                                     |
| Combiner Function        | A reducer-like function that pre-aggregates same-key data before it crosses the network, reducing data transfer                                 |
| Combinable Reducer       | A reduce function whose output has the same shape as its input, allowing it to double as a combiner                                             |
| Materialized View        | A stored, precomputed map-reduce result, reused across multiple downstream queries                                                              |
| Incremental Map-Reduce   | Recomputing only the parts of a map-reduce result affected by new or changed data, instead of starting from scratch                             |
| Key-Value Store          | A simple hash-table-style database where all access is via a primary key, and the value is an opaque blob                                       |
| Bucket                   | A flat namespace (in Riak) used to segment keys and avoid key collisions between different types of data                                        |
| Domain Bucket            | A bucket dedicated to a specific object type, letting the client driver handle serialization automatically                                      |
| N, R, W                  | Replication factor, read quorum, and write quorum — the tunable knobs controlling consistency and availability in a distributed key-value store |

### 4.2 Comparison Tables to Memorize

#### Map vs. Reduce Recap:

| Stage  | Operates On                 | Produces                              |
|--------|-----------------------------|---------------------------------------|
| Map    | A single aggregate          | Key-value pairs                       |
| Reduce | All values for a single key | One combined/summarized value per key |

#### Map-Reduce Optimizations Recap:

| Technique              | Purpose                                                                           |
|------------------------|-----------------------------------------------------------------------------------|
| Partitioning           | Enables multiple reducers to run in parallel, each on its own key-group           |
| Combining              | Reduces network data transfer by pre-summarizing same-key data before the shuffle |
| Incremental Map-Reduce | Avoids full recomputation by reusing prior results when changes are additive      |

#### Key-Value Databases — When to Use vs. When NOT to Use:

|  |
|--|
|  |
|--|

| When to Use                                                          | When NOT to Use                                                         |
|----------------------------------------------------------------------|-------------------------------------------------------------------------|
| Storing session information, keyed by session ID                     | Data has meaningful relationships that must be queried or traversed     |
| Storing user profiles and preferences, keyed by user ID              | Need multi-operation transactions with rollback across several keys     |
| Storing shopping cart data, keyed by user ID, with high availability | Need to query or filter based on the contents of the value, not the key |
| Any workload needing fast, single-key GET/PUT/DELETE access          | Need to operate on multiple keys at once (set-based operations)         |